Dependencies

Backend

Dependency	Purpose
django	Backend framework and core functionality
django-cors-headers	Allows for communication between the frontend and backend
django-rest-framework	Toolkit to create RESTful API
django-rest-framework-simplejwt	Ensures secure API endpoints user authentication
numpy	Processes large amounts of tabular data (e.g. csv files) to analyze
pandas	Performs data analysis on subscription usage data
requests	Interacts with external APIs like Rescuetime and OneSignal
pytz	Handles correct datetimes for subscription notifications or expiry

Frontend

Dependency	Purpose
vue	Frontend framework
vue-router	Routes pages on website
@vite	Core for fast build processes
@vitejs/plugin-vue	Integration with vue
Primevue	UI components and theming
TailwindCSS	CSS styling and responsive design
tailwindcss/prime-ui	Customize UI components
Axios	Interact with APIs and fetch data
apexcharts	Create customized charts for statistics

File Structure

Self explanatory names and file structure

```
|  README.md
|  user_documentation.txt
```

Backend

```
|   .env
|   db.sqlite3
|   manage.py
|   requirements.txt
|
+---api
|   |   admin.py
|   |   apps.py
|   |   authentication.py
|   |   middleware.py
|   |   models.py
|   |   notifications.py
|   |   screentime.py
|   |   serializers.py
|   |   services.py
|   |   tests.py
|   |   urls.py
|   |   utils.py
|   |   __init__.py
|   |
|   \---views
|           analytics_views.py
|           cron_views.py
|           model_views.py
|           user_views.py
|           __init__.py
|
\---backend
        asgi.py
        settings.py
        urls.py
        wsgi.py
        __init__.py
```

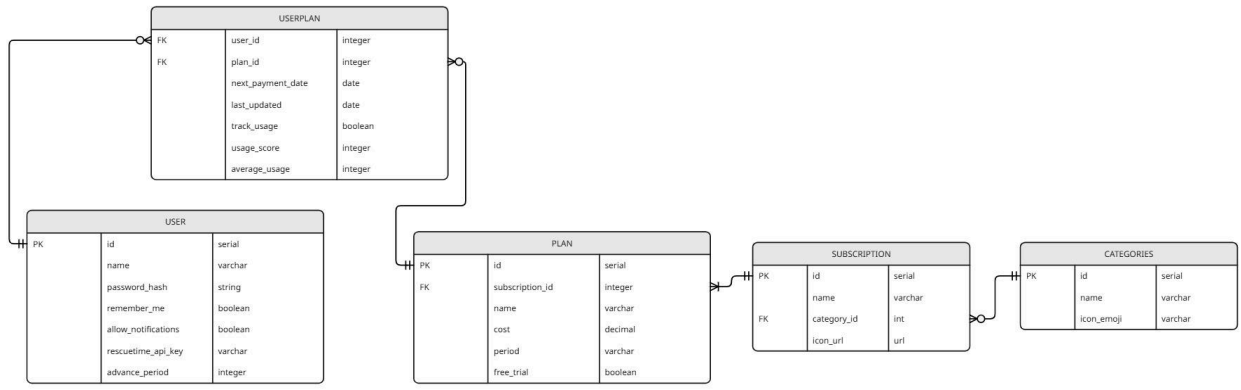
Frontend

```
|   .editorconfig
|   .prettierrc.json
|   eslint.config.js
|   index.html
|   jsconfig.json
|   package-lock.json
|   package.json
|   README.md
|   vite.config.js
|
+---public
|       favicon.ico
|       OneSignalSDKWorker.js
|
\---src
    |   App.vue
    |   main.js
    |
    +---assets
    |       base.css
    |       tailwind.css
    |
    +---components
    |       CardWrapper.vue
    |       CategoryStats.vue
    |       CategoryTable.vue
    |       DetailsCard.vue
    |       FieldWrapper.vue
    |       MobileOverlay.vue
    |       Sidebar.vue
    |       SpendingStats.vue
    |       SubscriptionList.vue
    |       SucessCard.vue
    |       Topbar.vue
    |       UserPlanCard.vue
    |       UserPlanList.vue
    |
    +---composables
    |       index.js
    |       useHelpers.js
    |       useSubscriptionManager.js
    |       useUser.js
    |       useValidateForms.js
    |
```

```
+---layouts
|       MainLayout.vue
|       PublicLayout.vue
|
+---router
|       index.js
|       routes.js
|
+---services
|       oneSignalService.js
|
\---views
        AddPlanView.vue
        BudgetView.vue
        EditPlanView.vue
        EditUserPlanView.vue
        HomeView.vue
        LoginView.vue
        MyListView.vue
        NotFoundView.vue
        ProfileView.vue
        RegisterView.vue
        SettingsView.vue
        SharedListView.vue
```

ORM Database Model

For permanent and efficient storage, I chose a **SQLite database**. This was a good fit due to the relational nature of my data, which required reliable and structured operations like notifying payment dates or gathering analytics from user subscriptions.



Rather than writing raw SQL, I used **Django's ORM (Object-Relational Mapping)** which mapped database interactions to classes, aligning with Python's **OOP** principles. For example, I could define the **many-to-many associations** between user and plan tables through a "foreign key" field, and encapsulate logic in custom methods to control access. This ensured the codebase was more maintainable and reusable to fulfill my client's need for forward-compatibility.

```
100 class UserPlan(models.Model):
101     user = models.ForeignKey(User, on_delete=models.CASCADE, related_name="user_plans")
102     plan = models.ForeignKey(Plan, on_delete=models.CASCADE)
103
104     payment_date = models.DateField()
105     last_updated = models.DateField(null=True, blank=True)
106     total_spent = models.IntegerField(default=0)
107
108     track_usage = models.BooleanField(default=False)
109     usage_score = models.IntegerField(default=1, choices=USAGE_SCORE_CHOICES)
110     average_usage = models.IntegerField(default=0)
111
112     def update_payment_date(self):
113         """Updates payment date to always be the most recent (next date)"""
114         today = date.today()
115
116         if self.payment_date < today:
117             if self.plan.free_trial:
118                 return
119
120             if not self.plan.period:
121                 raise ValueError("Period must be set for the plan.")
122
123             self.payment_date += timedelta(days=self.plan.period)
124             self.total_spent += self.plan.cost
125             self.last_updated = today
126
127             self.save()
```

Using **inheritance** from Django's `model.Model` class, I could use built-in methods for query and filtering.

```
152    user_plans = UserPlan.objects.filter(user=user, cost__lte=cost_max)
```

MVT Architecture

The use of Django's **Model View Template** framework also allowed for clean separation of my app's logic. For example, application logic such as parsing and validating query parameters resided in views and serializers, while business logic was handled in services.

```python
from ..models import Plan, UserPlan
from ..serializers import UserPlanSerializer, PeriodQueryParamSerializer
from ..services import SpendingCalculator, AverageSpendingCalculator


class AverageSpendingPerPeriod(APIView):
    """Gets normalized total spending for each period"""

    def get(self, request):
        user = request.user

        try:
            # Validate params using the serializers (application layer)
            serializer = PeriodQueryParamSerializer(data=request.query_params)
            serializer.is_valid(raise_exception=True)
            period_param = serializer.validated_data.get("period", None)

            # Determine target periods
            if period_param:
                target_periods = [(period_param, Plan.Period.get_label(period_param))]
            else:
                target_periods = Plan.Period.choices

            # Retrieve user plans (data access layer)
            user_plans = UserPlan.objects.filter(user=user)

            # Calculate average results using services (business layer)
            calculator = AverageSpendingCalculator(user_plans)
            results = calculator.calculate_averages(target_periods)

            return Response(results)

        except ValidationError as e:
            return Response({"error": str(e)}, status=400)


```

Fetching Screen Time Data

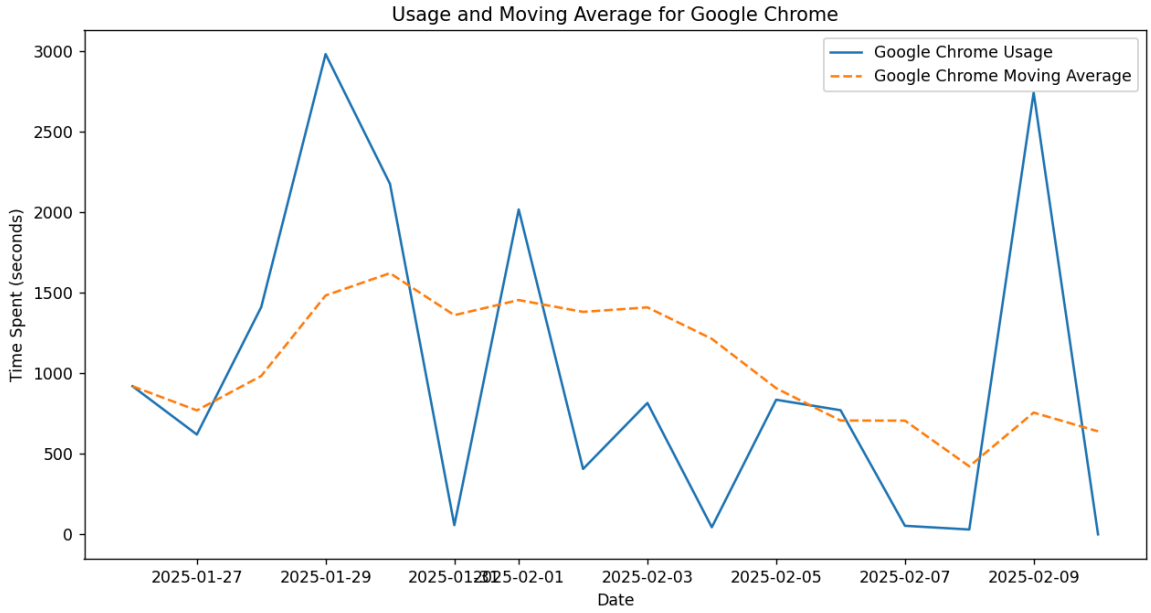
I used the Rescuetime API to fetch usage data over a specific period of time. I used list comprehension to format the url with query params as key value pairs. As this was in a raw character string format, `stringIO` converted this API response into a file, allowing it to be read by the `pandas` library and parsed as a `.csv` file.

```
8   def fetch_data(api_key, start_date, end_date):
9       """
10      Fetch RescueTime's summary data analytics from start_date and end_date.
11      Dates is in YYYY-MM-DD form.
12      """
13      url = "https://www.rescuetime.com/anapi/data"
14      params = {
15          "key": api_key,
16          "perspective": "interval",
17          "resolution_time": "day",
18          "restrict_begin": start_date,
19          "restrict_end": end_date,
20          "format": "csv",
21      }
22
23      # Format url with query params
24      full_url = url + "?" + "&".join([f"{key}={value}" for key, value in params.items()])
25
26      response = requests.get(url, params=params)
27      if response.ok:
28          data = StringIO(response.text) # cnverts response character stream into a file-object
29          df = pd.read_csv(data) # parses file as a csv dataframe
30
31          # Preprocess data
32          df["Date"] = pd.to_datetime(df["Date"]) # convert date strings to datetime type
33          df = df.rename(columns={"Time Spent (seconds)": "Time"})
34          df = df.groupby(["Date", "Activity"])["Time"].sum().reset_index() # merge duplicates
35          df = df.pivot(index="Date", columns="Activity", values="Time") # organize by date
36          df = df.fillna(0) # assume 0 for missing cells
37
38          return df
39      else:
40          response.raise_for_status()
41
```

Time Series Decomposition

In order to identify unused subscriptions, I calculated a custom usage score metric based on **time-series decomposition**. As screen time usage fluctuates and is often very seasonal -- e.g. work-related products seeing high use during weekdays but little on weekends -- using simple short term data would trigger false flags, leading to unnecessary notifications disrupting effective decision making and user experience.

Thus, I used time-series decomposition, which could identify more meaningful long-term trends and usage patterns. This method smoothed out short term fluctuations and cycles by taking a rolling average of the usage data with a window size of 7 days, which was around the time of a typical cycle.



To efficiently calculate the moving average, I kept a rolling sum of the time spent while iterating through the data by date. If a data point slid out the window, I would subtract its value from the sum. The moving average of the last `window_size` data points could then be calculated as `sum / window_size`, ensuring a linear time complexity.

```
43  def _calculate_moving_average(data, window_size):
44      """
45      Calculates the moving average for a given list of data using a rolling window.
46      """
47      moving_averages = []
48      current_sum = 0  # sum of the current window
49
50      for i in range(len(data)):
51          current_sum += data[i]
52
53          # If boundary is hit, subtract the element that's leaving the window
54          if i >= window_size:
55              current_sum -= data[i - window_size]
56
57          # Calculate the moving average (for windows that have full window size)
58          moving_avg = current_sum / min(i + 1, window_size)  # adjust for the first few elements
59          moving_averages.append(moving_avg)
60
61      return moving_averages
62
```

Once the moving average was computed for each subscription, this was compared against an unused “threshold” value and a trend line was created. If the subscription usage was lower than the threshold or the trend slope was decreasing significantly, the subscription would be flagged as unused and users would be notified.

```python
def calculate_usage(subscription_name, df, threshold=300, window_size=7, trend_period=14, trend_threshold=0.8):
    """
    Grades a subscription's usage from 1 to 10 based on screentime data.
    It analyzes usage patterns over time, including a moving average
    for more stable results and compares with trends in historical data to detect if it's downward.
    """

    if subscription_name not in df.columns:
        print(f"Warning: '{subscription_name}' not found in the data.")
        return 0

    if len(df) < trend_period:
        print(f"Warning: Not enough data to calculate the moving average for {subscription_name}.")
        return 0

    # Call to calculate the moving average with an internal method
    df[f"{subscription_name}_MA"] = _calculate_moving_average(df[subscription_name], window_size)

    # Get the most recent and older moving averages
    recent_ma = df[f"{subscription_name}_MA"].iloc[-1]  # most recent moving average
    older_ma = df[f"{subscription_name}_MA"].iloc[-trend_period]  # moving average for trend_period days ago

    # Calculate the base score (0 to 10) based on the moving average and threshold
    if recent_ma >= threshold:
        base_score = 10  # well above the threshold
    else:
        base_score = (recent_ma / threshold) * 10  # normalize to 0-10

    # Reduce score if the subscription is trending downward
    if recent_ma < older_ma * trend_threshold:
        penalty = (1 - (recent_ma / older_ma)) * 5  # deduct up to 5 points
        base_score -= penalty

    # Final clipping between 1 and 10
    usage_score = max(1, min(10, round(base_score)))

    return usage_score
```

Dynamic Programming

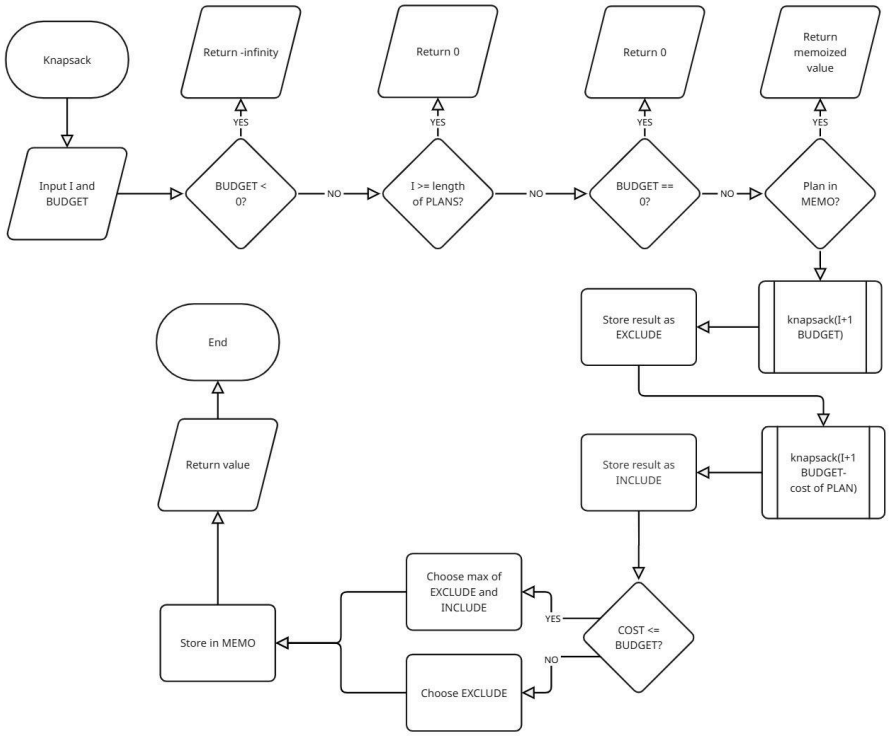
To program the “set budget” function, I made use of **dynamic programming** and **recursion** to fit the user’s subscriptions within the user-defined budget, while prioritizing more “important” subscriptions.

This could be modelled as a **0-1 knapsack problem**, where given a fixed size “knapsack”, the goal is to pack various items with associated values and weights while maximizing total value. In this context, the subscriptions have:

- Cost - the “weight”
- Usage score - a reliable heuristic for “value,” as users who spent more time likely find it more valuable

The solution involves iterating through each plan in the list. At each step, the problem could split into the two paths, where an item was either included or excluded.
In the exclusion case, the budget remains the same and could simply move on to the next subscription. This was also the only case if the item cannot fit within the budget.
If an item was included, a subproblem knapsack could be run, with the new budget being the remaining space.

$$dp[i][j] = \begin{cases} dp[i-1][j], & \text{if } w[i] > j \\ \max\left(dp[i-1][j], dp[i-1][j-w[i]] + v[i]\right), & \text{if } w[i] \leq j \end{cases}$$



```python
def _knapsack(i, remaining, plans, memo):
    """Recursive 01 knapsack, using 2d dictionary of tuples (pairs) as key to save memory (~O
    if remaining < 0:
        return -float("inf")
    if i >= len(plans) or remaining == 0:
        return 0
    if (i, remaining) in memo:
        return memo[(i, remaining)]

    current = plans[i]
    # Explore exclusion path
    exclude = _knapsack(i + 1, remaining, plans, memo)

    # Explore inclusion path
    include = current["usage_score"] + _knapsack(
        i + 1, remaining - current["scaled_cost"], plans, memo
    )

    # Choose maximum value between the two and store in memo
    max_val = max(exclude, include) if current["scaled_cost"] <= remaining else exclude
    memo[(i, remaining)] = max_val
    return max_val


```

To increase efficiency, I used **memoization** with a **2d array-hashmap**. Memoization stores the result for the current budget and the current subscription index, allowing reuse for solved subproblems, drastically reducing the number of computations. As the budget could grow very large, and a 2d memo array would grow in O(N*W), a hashmap was used to reduce the space. This was keyed with a combination of the budget and plan index.

As the algorithm only works with whole numbers otherwise the search space would be too large, I rounded the budgets. As well, I used **lambda expressions** to sort the plans by a computed usage score to cost ratio, which improved runtime on larger inputs by pruning suboptimal paths earlier.

```python
def budget_plans(user_plans, original_budget):
    """
    Finds optimat set of subscriptions to include w/in a given budget
    Checks combinations of plan costs (weights) while maximizing usage score (value)
    Returns list of plan ids included in the budget
    """
    # Preprocess and sort plans by cost to usage score ratio (may prune suboptimal paths sooner),
    # and rounds costs to use for the dp memoization table
    processed = sorted(
        (
            {
                "id": p["id"],
                "usage_score": p["usage_score"],
                "scaled_cost": round(p["cost"]),
                "value_density": p["usage_score"] / max(p["cost"], 1e-9), # Avoid divide by zero/floating
            }
            for p in user_plans
        ),
        key=lambda x: (-x["value_density"], x["scaled_cost"]),
    )

    # Initialize memoization table and solve (build memo table)
    budget = round(original_budget)
    memo = {}
    max_score = _knapsack(0, budget, processed, memo)

```

In order to find the plans, I then backtracked through the constructed memoization table to reconstruct the optimal selections. This involved calculating the score if a plan was included by checking the memoization map, and checking if it equaled the optimal score to see if it was included.

```python

    # Backtracks through memo table to reconstruct optimal selections
    selected = []
    remaining_budget = budget

    for idx, plan in enumerate(processed):
        cost = plan["scaled_cost"]
        if remaining_budget < cost:
            continue

        # Check if current plan was included in optimal solution
        with_current = (
            memo.get((idx + 1, remaining_budget - cost), 0) + plan["usage_score"]
        )
        if memo.get((idx, remaining_budget), 0) == with_current:
            selected.append(plan["id"])
            remaining_budget -= cost

    return selected

```

Cookie-based Authentication (“Remember me”)

My client, as a financial advisor, handles various online products and payment plans that contain confidential personal and financial information, (financial - payment plans, subscription data; personal - screen time usage). Therefore, an **authentication system** was required to ensure security and prevent unauthorized access from individuals or external threats.
I chose **JWT tokens** as they were lightweight and secure. For a remember me functionality, I used **HTTPOnly cookies** to store the user’s session information. This protected session data from cross-site scripting attacks as HTTPonly cookies cannot be accessed by frontend scripts.

```
4   class CookieJWTAuthentication(JWTAuthentication):
5       def authenticate(self, request):
6           token = request.COOKIES.get("access_token")
7
8           if not token:
9               return None
10          try:
11              validated_token = self.get_validated_token(token)
12          except AuthenticationFailed as e:
13              raise AuthenticationFailed(f"Token validation failed: {str(e)}")
14
15          try:
16              user=self.get_user(validated_token)
17              return user, validated_token
18          except AuthenticationFailed as e:
19              raise AuthenticationFailed(f"Error retrieving user: {str(e)}")
```

As well, I programmed a custom **backend middleware** which would intercept each request and if the remember me cookie was attached, refresh the token if it was close to expired. This ensured continuous access

```python
class TokenRefreshMiddleware:
    def __init__(self, get_response):
        self.get_response = get_response

    def __call__(self, request):
        response = self.get_response(request)

        # Retrieve the access token from the HTTPOnly cookie
        access_token = request.COOKIES.get("access_token")
        refresh_token = request.COOKIES.get("refresh_token")
        remember_me = request.COOKIES.get("remember_me") == "True"

        # Check if user has remember me or is properly authenticated
        if not (remember_me and access_token and refresh_token):
            return response

        try:
            # Parse the access token and check expiration
            access = AccessToken(access_token)
            token_expiry = datetime.fromtimestamp(access["exp"])

            # Check if token is about to expire
            if datetime.now() > token_expiry - timedelta(minutes=10):
                # Refresh the token
                refresh = RefreshToken(refresh_token)
                new_access = refresh.access_token

                # Sets cookies
                response.set_cookie(
                    key="access_token",
                    value=str(new_access),
                    httponly=True,
                    secure=True,
                    samesite="None",
                    max_age=new_access.lifetime,
                )

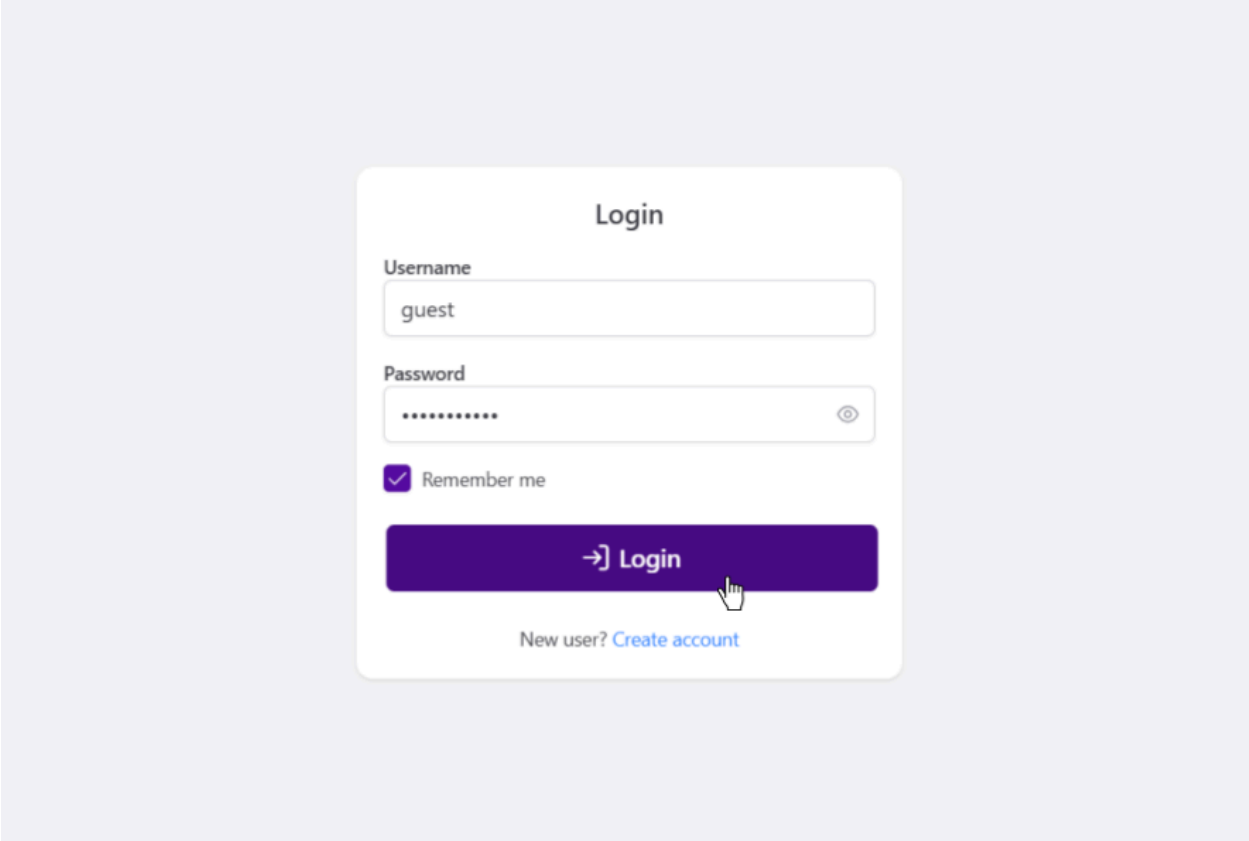
                response.set_cookie(
                    key="refresh_token",
                    value=str(refresh),
                    httponly=True,
                    secure=True,
                    samesite="None",
                    max_age=refresh.lifetime,
                )
        except Exception as e:
            pass # Handle invalid/expired tokens gracefully

        return response

```

User Authentication State

This was saved into a global user state, which was created using Vue's reactivity object. This allowed only a one time api call to fetch user information. Subsequently, any calls could instead import from a userStore composable, and fetch data from there.



On the frontend, I added **navigation guards** to authorized pages by automatically verifying the user's state and redirecting them to the login page each new page. This ensured a better user experience as unauthorized or expired users would receive immediate feedback, and more security as users could not gain access to unauthorized areas.

```
13  router.beforeEach(async (to, from, next) => {
14    if (userStore.loggedIn) {
15      // Already authenticated, redirect to home or previous route
16      if (to.name === 'login' || to.name === 'register') {
17        return next(from.path || '/')
18      }
19    } else {
20      if (to.meta.requiresAuth) {
21        try {
22          const response = await axios.post('/api/auth/verify/') // Verify user's session (access token)
23          if (response.status === 200) { // Valid session, proceed to requested route
24            userStore.login()
25            return next()
26          }
27        } catch (error) { // Token is expired/invalid, logout
28          userStore.logout()
29          return next('/login')
30        }
31      }
32    }
33
34    next() // If route doesn't require auth or user is logged in
35  })
36  export default router
37
```

Asynchronous Programming

As communication with the backend could take long wait times, asynchronous programming was used to ensure a smooth and timely user experience. Fetching large amounts of data such as subscription plans were done with Javascript's "async" and "await" keywords, which allowed the website to perform other tasks while waiting for the API to return data.

```
// Fetch Plans with applied filters
const fetchPlans = async () => {
  try {
    const { category, costRange, period } = filters.value

    if (costRange[0] > costRange[1]) filters.value.costRange = [costRange[1], costRange[0]]

    let queryParams = ''

    queryParams += `cost_min=${costRange[0]}&`
    queryParams += `cost_max=${costRange[1]}&`
    if (category) queryParams += `category_id=${category}&`
    if (period) queryParams += `period=${period.toLowerCase()}&`

    queryParams = queryParams ? queryParams.slice(0, -1) : '' // Remove trailing "&"
    console.log(queryParams);
    const response = await axios.get(`/api/plans/?${queryParams}`)
    plans.value = response.data
  } catch (error) {
    console.error('Error fetching plans:', error)
  }
}
```

Environment Variables and External APIs

In order to provide more security, API keys were stored in a .env file.

```python
import os
import requests
from dotenv import load_dotenv # Loads environment variables

load_dotenv()

ONESIGNAL_API_KEY = os.getenv("ONESIGNAL_API_KEY")
ONESIGNAL_APP_ID = os.getenv("ONESIGNAL_APP_ID")
ONESIGNAL_API_URL = "https://api.onesignal.com"


def check_user_subscription(user_id):
    """ Checks if a user is resgistered and subscribed to notifications (via OneSignal external_id tag) """
    url = f"{ONESIGNAL_API_URL}/apps/{ONESIGNAL_APP_ID}/users/by/external_id/{user_id}"
    headers = {"Authorization": f"Basic {ONESIGNAL_API_KEY}"}

    try:
        response = requests.get(url, headers=headers)
        response.raise_for_status()  # Raise an exception for HTTP errors
        user_data = response.json()

        # Check if the user is subscribed
        return user_data.get("subscriptions", [{}])[0].get("notification_types") == 1
    except requests.exceptions.RequestException as e:
        print(f"Failed to fetch user data for user {user_id}: {e}")
        return False

```